

DATABASE

MUHAMMAD TAHA

22F-3277

BCS-3C

Q1

```
#include<iostream>
#include<fstream>
#include<string>
using namespace std;
class Course
{
public:
    string name;
    char grade;
    Course(string name = "", char grade = 'I', float gpa = 0) : name(name),
grade(grade){}
    void setName(string name)
    {
        this->name = name;
    }
    void setGrade(char grade)
    {
        this->grade = grade;
    }
    string getCourse()
    {
        return name;
    }
    char getGrade()
    {
        return grade;
    }
};
class Student
{
public:
    string name;
    int id;
    Course coursesEnrolled[5];
    float cgpa;
    Student(string name = "", int id = 0, float gpa = 0) : name(name), id(id), cgpa(gpa)
{}
    void setName(string name)
    {
        this->name = name;
    }
    void setId(int id)
    {
        this->id = id;
    }
    void setGPA(float gpa)
    {
        this->cgpa = gpa;
    }
    void enrollInCourse()
    {
        string courseName;
        char courseGrade;
        for (int i = 0; i < 5; i++)
        {
            cout << "Enter Course " << i + 1 << " = ";
            cin >> courseName;

            cout << "Enter " << courseName << "'s Grade = ";
            cin >> courseGrade;

            coursesEnrolled[i].setName(courseName);
            coursesEnrolled[i].setGrade(courseGrade);
        }
    }
}
```

```

void writeStudentInFile()
{
    ofstream writeInFile;

    writeInFile.open("StudentsDetail.txt", ios::app);
    if (writeInFile.is_open())
    {
        writeInFile << name << " " << id;
        for (int i = 0; i < 5; i++)
        {
            writeInFile << coursesEnrolled[i].getCourse() << " " <<
coursesEnrolled[i].getGrade();
        }
        writeInFile << " " << cgpa << endl;
    }
    else
        cout << "File not open\n";

    writeInFile.close();
}
};
class IMS
{
    Student *students;
    int totalStudent;
public:
    IMS(int totalStudent = 0)
    {
        this->totalStudent = countStudentInFile();
        students = new Student[this->totalStudent];
        loadStudentsFromFile();
    }
    int countStudentInFile()
    {
        ifstream read;
        int studentCount = 0;
        string line;
        read.open("StudentsDetail.txt");
        if (read.is_open())
        {
            while (getline(read, line))
                studentCount++;
        }
        else
            cout << "File not opened for counting students\n";

        read.close();

        return studentCount;
    }
    void loadStudentsFromFile()
    {
        ifstream readFromFile;
        int i = 0;
        string name;
        int id;
        string courseName;
        char grade;
        float gpa;

        readFromFile.open("StudentsDetail.txt");
        if (readFromFile.is_open())
        {
            while (readFromFile >> name >> id)
            {
                students[i].name = name;
                students[i].id = id;
                for (int c = 0; c < 5; c++)

```

```

        {
            readFromFile >> courseName >> grade;
            students[i].coursesEnrolled[c].setName(courseName);
            students[i].coursesEnrolled[c].setGrade(grade);
        }
        readFromFile >> gpa;
        students[i].cgpa = gpa;
        i++;
    }
}
else
    cout << "File not open for loading\n";

    readFromFile.close();
}
void printAllStudentDetail()
{
    for (int i = 0; i < totalStudent; i++)
    {
        cout << "-----";
        cout << "\nName: " << students[i].name;
        cout << "\tID: " << students[i].id;
        cout << "\nCourses: \n";
        for (int c = 0; c < 5; c++)
            cout << "\t" << students[i].coursesEnrolled[c].name << " " <<
students[i].coursesEnrolled[c].grade << "\n";
        cout << "\nCGPA: " << students[i].cgpa << "\n";
    }
}
void addStudent()
{
    string name;
    int id = 0;
    int gpa;
    cout << "Enter Name: ";
    cin >> name;
    cout << "Enter ID: ";
    cin >> id;
    while (id > 9999 || id < 999 )
    {
        cout << "Enter 4 digit id: ";
        cin >> id;
    }
    cout << "Enter CGPA: ";
    cin >> gpa;
    while (gpa > 4 || gpa < 1)
    {
        cout << "Enter between 1-4";
        cin >> gpa;
    }

    Student newStudent;
    newStudent.setName(name);
    newStudent.setId(id);
    newStudent.enrollInCourse();
    newStudent.writeStudentInFile();
    updateSystem();
}
void updateSystem()
{
    this->totalStudent = countStudentInFile();
    students = new Student[this->totalStudent];
    loadStudentsFromFile();
}
void removeStudent(int removeID)
{
    ifstream readFromFile;

```

```

string line;
int lineNoOfStudentToDelete = 1;
string temp, name;
int id;
readFromFile.open("StudentsDetail.txt");
bool found = false;
if (readFromFile.is_open())
{
    while (readFromFile >> name >> id)
    {
        getline(readFromFile, line);
        if (id == removeID)
        {
            found == true;
            readFromFile.close();
        }
        else
            lineNoOfStudentToDelete++;
    }
}
else
    cout << "File not open for removing student\n";

if (found)
{
    string file = "StudentsDetail.txt";
    //removing LOGIN DETAILS by removing the line
    deleteFromFile(file, lineNoOfStudentToDelete);
    updateSystem();
}
else
    cout << "Student not Found\n";
}

void deleteFromFile(const string& filePath, int deleteLineNo)
{
    ifstream readFile(filePath);
    ofstream writeTempFile("temp.txt");
    string line;
    int currentLineNo = 0;

    while (getline(readFile, line))
    {
        ++currentLineNo;

        if (currentLineNo != deleteLineNo)
        {
            writeTempFile << line << endl;
        }
    }

    readFile.close();
    writeTempFile.close();

    if (remove(filePath.c_str()) != 0)
    {
        cout << "Error deleting the " << filePath << " file." << endl;
        return;
    }

    if (rename("temp.txt", filePath.c_str()) != 0)
        cerr << "Error renaming the " << filePath << " file." << endl;
}

void generateCGPARReport(float gpa)
{
    cout << "Report of Student with CGPA >= " << gpa;
    for (int i = 0; i < totalStudent; i++)
    {
        if (students[i].cgpa >= gpa)

```

```

        {
            cout << "-----";
            cout << "\nName: " << students[i].name;
            cout << "\tID: " << students[i].id;
            cout << "\nCourses: \n";
            for (int c = 0; c < 5; c++)
                cout << "\t" << students[i].coursesEnrolled[c].name << " "
<< students[i].coursesEnrolled[c].grade << "\n";
            cout << "\nCGPA: " << students[i].cgpa << "\n";
        }
    }

    void updateID(int old, int newID)
    {
    }

};

int main()
{
    IMS flex;
    int choice = 0;
    while(1)
    {
        system("cls");
        cout << "\n1. Add Students\n";
        cout << "\n2. Delete Students\n";
        cout << "\n3. Update Name\n";
        cout << "\n4. Print All Students\n";
        cout << "\n5. Generate Report\n";
        cout << "\n6. Exit\n";
        cin >> choice;

        switch (choice)
        {
            case 1:
                flex.addStudent();
                break;
            case 2:
            {
                int id;
                cout << "Enter Id to remove: ";
                cin >> id;
                flex.removeStudent(id);
                break;
            }
            case 3:
            {
                int oldId, uID;
                cout << "Enter Old Name: ";
                cin >> oldId;
                cout << "Enter updated Name: ";
                cin >> uID;
                flex.updateID(oldId, uID);
            }
            break;
            case 4:
                flex.printAllStudentDetail();
                break;
            case 5:
            {
                float gpa;
                cout << "Enter minimum cgpa: ";
                cin >> gpa;
                flex.generateCGPAReport(gpa);
                break;
            }
            case 6:
                return 0;
        }
    }
}

```

```
    }  
    system("pause");  
}  
return 0;  
}
```

```
1. Add Students  
2. Delete Students  
3. Update Name  
4. Print All Students  
5. Generate Report  
6. Exit  
4  
-----  
Name: Taha      ID: 3277  
Courses:  
    Data B  
    COAL A  
    OOP B  
    PF B  
    OS A  
  
CGPA: 3.2  
-----  
Name: Assadad  ID: 4355  
Courses:  
    Data F  
    COAL D  
    OOP B  
    PF B  
    OS C  
  
CGPA: 2.5  
Press any key to continue . . . |
```

```

1. Add Students
2. Delete Students
3. Update Name
4. Print All Students
5. Generate Report
6. Exit
4
-----
Name: Taha      ID: 3277
Courses:
    Data B
    COAL A
    OOP B
    PF B
    OS A

CGPA: 3.2
-----
Name: Assadad   ID: 4355
Courses:
    Data F
    COAL D
    OOP B
    PF B
    OS C

CGPA: 2.5
-----
Name: Qasim     ID: 6565
Courses:
    .9 D
    ata a
    oop b
    pf c
    la d

CGPA: 0
Press any key to continue . . . |

```

```

1. Add Students
2. Delete Students
3. Update Name
4. Print All Students
5. Generate Report
6. Exit
5
Enter minimum cgpa: 2
Report of Student with CGPA >= 2-----
Name: Taha      ID: 3277
Courses:
    Data B
    COAL A
    OOP B
    PF B
    OS A

CGPA: 3.2
-----
Name: Assadad   ID: 4355
Courses:
    Data F
    COAL D
    OOP B
    PF B
    OS C

CGPA: 2.5
Press any key to continue . . . |

```

